Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 8ο
Ψηφιακά Συστήματα VLSI

2η Εργαστηριακή Άσκηση

Σχεδίαση Μονάδων Υλικού με την Τεχνική Pipelining

Μέλη Ομάδας

Καβαδάκης Σωτήρης 03122035
Κοντοπίδης Δημήτρης 03122025

Αθήνα, Μάρτιος 2026

Ερώτημα 1: Σύγχρονος Πλήρης Αθροιστής (Synchronous Full Adder)

Περιγραφή Κυκλώματος

Ο Σύγχρονος Πλήρης Αθροιστής (Synchronous Full Adder) είναι ένα ακολουθιακό κύκλωμα που εκτελεί πρόσθεση τριών bits (δύο bits εισόδου και ένα κρατούμενο εισόδου) με τις εξόδους να καταχωρούνται σε flip-flops στη θετική ακμή του ρολογιού. Σε αντίθεση με τον συνδυαστικό Full Adder, οι έξοδοι ενημερώνονται μόνο στη θετική ακμή του σήματος ρολογιού, εισάγοντας καθυστέρηση ενός κύκλου.

Υλοποιήθηκε με περιγραφή συμπεριφοράς (Behavioral) χρησιμοποιώντας process με ευαισθησία στο ρολόι.

Είσοδοι:

- i_clk: Σήμα ρολογιού
- i_a: Πρώτο bit εισόδου
- i_b: Δεύτερο bit εισόδου
- i_cin: Κρατούμενο εισόδου

Έξοδοι:

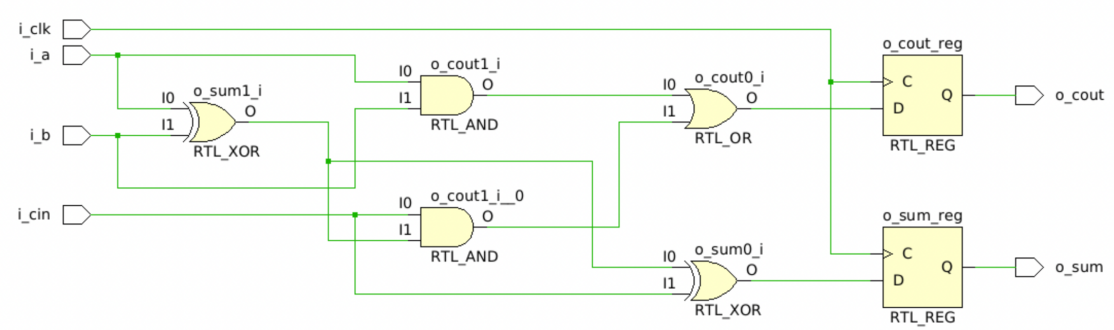
- o_sum: Άθροισμα ($S = A \oplus B \oplus C_{in}$)
- o_cout: Κρατούμενο εξόδου ($C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$)

Ο πίνακας αλήθειας του Synchronous Full Adder:

i_a	i_b	i_cin	o_sum	o_cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Πίνακας 1Πίνακας αλήθειας Synchronous Full Adder

a) Σχεδιάγραμμα Λειτουργίας (RTL Schematic)



Σχήμα RTL σχηματικό του Synchronous Full Adder

Το RTL σχηματικό αποτελείται από:

- Μία πύλη **XOR** για τον υπολογισμό του ενδιάμεσου $A \oplus B$
- Δύο πύλες **AND** για τους όρους $A \cdot B$ και $C_{in} \cdot (A \oplus B)$
- Μία πύλη **OR** για τον υπολογισμό του κρατούμενου C_{out}
- Μία πύλη **XOR** για τον υπολογισμό του τελικού αθροίσματος $(A \oplus B) \oplus C_{in}$
- Δύο **RTL_REG** (flip-flops) που καταχωρούν τα o_sum και o_cout στη θετική ακμή του ρολογιού

b) Κώδικας VHDL - Behavioral Model

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sync_full_adder is
    Port (
        i_clk  : in STD_LOGIC;
        i_a    : in STD_LOGIC;
        i_b    : in STD_LOGIC;
        i_cin  : in STD_LOGIC;
        o_sum  : out STD_LOGIC;
        o_cout : out STD_LOGIC
    );
end sync_full_adder;

architecture Behavioral of sync_full_adder is
begin
```

```
    process(i_clk)
    begin
        if rising_edge(i_clk) then
            o_sum <= (i_a XOR i_b XOR i_cin);
            o_cout <= (i_a AND i_b) OR (i_cin AND (i_a XOR i_b));
        end if;
    end process;
end Behavioral;
```

Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity sync_full_adder_tb is
end entity;

architecture tb of sync_full_adder_tb is

    component sync_full_adder is
        Port (
            i_clk       : in  STD_LOGIC;
            i_a         : in  STD_LOGIC;
            i_b         : in  STD_LOGIC;
            i_cin       : in  STD_LOGIC;
            o_sum       : out STD_LOGIC;
            o_cout      : out STD_LOGIC
        );
    end component;

    signal i_Clk         : std_logic := '0';
    signal i_A           : std_logic := '0';
    signal i_B           : std_logic := '0';
    signal i_Carry_in    : std_logic := '0';
    signal o_Sum         : std_logic;
    signal o_Carry_out   : std_logic;

    constant CLK_PERIOD : time := 10 ns;

begin

    DUT: sync_full_adder
        port map (
            i_clk   => i_Clk,
            i_a     => i_A,
            i_b     => i_B,
```

```
            i_cin   => i_Carry_in ,
            o_sum   => o_Sum ,
            o_cout  => o_Carry_out
        );

    CLK_PROCESS: process
    begin
        i_Clk <= '0';
        wait for CLK_PERIOD / 2;
        i_Clk <= '1';
        wait for CLK_PERIOD / 2;
    end process;

    STIMULUS: process
    begin
        i_A <= '0'; i_B <= '0'; i_Carry_in <= '0';
        wait for CLK_PERIOD;
        assert (o_Sum = '0' and o_Carry_out = '0')
            report "FAIL: 0+0+0 expected Sum=0, Carry=0";

        i_A <= '0'; i_B <= '0'; i_Carry_in <= '1';
        wait for CLK_PERIOD;
        assert (o_Sum = '1' and o_Carry_out = '0')
            report "FAIL: 0+0+1 expected Sum=1, Carry=0";

        i_A <= '0'; i_B <= '1'; i_Carry_in <= '0';
        wait for CLK_PERIOD;
        assert (o_Sum = '1' and o_Carry_out = '0')
            report "FAIL: 0+1+0 expected Sum=1, Carry=0";

        i_A <= '0'; i_B <= '1'; i_Carry_in <= '1';
        wait for CLK_PERIOD;
        assert (o_Sum = '0' and o_Carry_out = '1')
            report "FAIL: 0+1+1 expected Sum=0, Carry=1";

        i_A <= '1'; i_B <= '0'; i_Carry_in <= '0';
        wait for CLK_PERIOD;
        assert (o_Sum = '1' and o_Carry_out = '0')
            report "FAIL: 1+0+0 expected Sum=1, Carry=0";

        i_A <= '1'; i_B <= '0'; i_Carry_in <= '1';
        wait for CLK_PERIOD;
        assert (o_Sum = '0' and o_Carry_out = '1')
            report "FAIL: 1+0+1 expected Sum=0, Carry=1";

        i_A <= '1'; i_B <= '1'; i_Carry_in <= '0';
        wait for CLK_PERIOD;
```

```
        assert (o_Sum = '0' and o_Carry_out = '1')
            report "FAIL: 1+1+0 expected Sum=0, Carry=1";

        i_A <= '1'; i_B <= '1'; i_Carry_in <= '1';
        wait for CLK_PERIOD;
        assert (o_Sum = '1' and o_Carry_out = '1')
            report "FAIL: 1+1+1 expected Sum=1, Carry=1";

        report "All tests passed!";
        wait;
    end process;

end architecture tb;
```

Ανάλυση Αποτελεσμάτων Testbench

Το testbench ελέγχει εξαντλητικά όλους τους $2^3 = 8$ δυνατούς συνδυασμούς εισόδου. Για κάθε συνδυασμό, οι είσοδοι εφαρμόζονται και μετά από έναν κύκλο ρολογιού (10 ns) ελέγχονται οι καταχωρημένες έξοδοι μέσω εντολών assert. Η καθυστέρηση ενός κύκλου αντικατοπτρίζει τη σύγχρονη φύση του κυκλώματος. Εφόσον δεν υπήρξε κάποιο assert error κατά την εκτέλεση, επιβεβαιώθηκε η ορθή λειτουργία.

c) Κρίσιμο Μονοπάτι (Critical Path)

Unconstrained Paths - NONE - clk - Setup

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock
Path 1	∞	2	3	2	i_a	o_sum_reg/D	1.932	1.132	0.800	∞	input port clock
Path 2	∞	2	3	2	i_cin	o_cout_reg/D	1.906	1.106	0.800	∞	input port clock

Σχήμα Κρίσιμο μονοπάτι του Synchronous Full Adder

Το κρίσιμο μονοπάτι του Synchronous Full Adder είναι από την είσοδο i_a προς τον καταχωρητή o_sum_reg/D με συνολική καθυστέρηση **1.932 ns** (Logic Delay: 1.132 ns, Net Delay: 0.800 ns). Το μονοπάτι διέρχεται από 2 επίπεδα λογικής (δύο πύλες XOR) και 3 routes. Πρόκειται για μονοπάτι τύπου input-to-register, καθώς οι είσοδοι δεν είναι καταχωρημένες.

Ερώτημα 2: Σύγχρονος Αθροιστής 4 bits με Pipeline

Περιγραφή Κυκλώματος

Ο Pipelined Αθροιστής 4 bits εκτελεί πρόσθεση δύο 4-bit αριθμών με κρατούμενο εισόδου, χρησιμοποιώντας την τεχνική pipeline. Η υλοποίηση βασίζεται σε τέσσερα instances του Synchronous Full Adder (Ερώτημα 1), συνδεδεμένα σε αλυσίδα ripple carry, με επιπλέον pipeline registers για την ευθυγράμμιση των δεδομένων.

Το κύκλωμα μπορεί να δέχεται ένα νέο ζεύγος εισόδων σε κάθε κύκλο ρολογιού και να παράγει ορθό αποτέλεσμα σε κάθε κύκλο μετά από αρχική καθυστέρηση $T_{latency} = 4$ κύκλους.

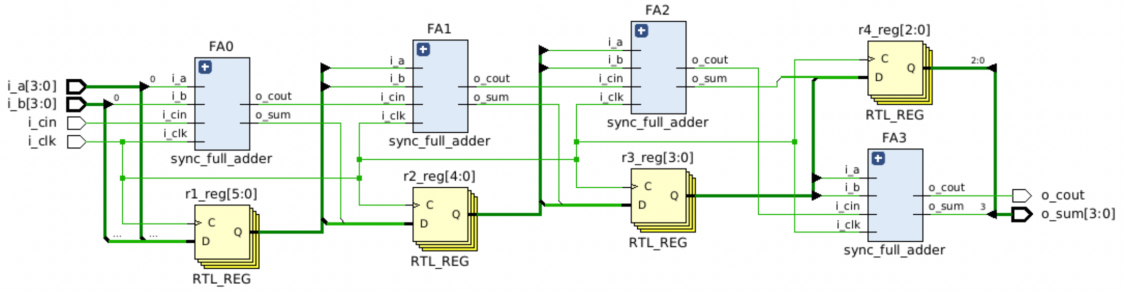
Είσοδοι:

- i_clk: Σήμα ρολογιού
- i_a(3:0): Πρώτος 4-bit αριθμός
- i_b(3:0): Δεύτερος 4-bit αριθμός
- i_cin: Κρατούμενο εισόδου

Έξοδοι:

- o_sum(3:0): 4-bit άθροισμα
- o_cout: Κρατούμενο εξόδου

a) Σχεδιάγραμμα Λειτουργίας (RTL Schematic)



Σχήμα RTL σχηματικό του Pipelined Adder 4 bits

Η αρχιτεκτονική pipeline χρησιμοποιεί τέσσερα στάδια, ένα για κάθε bit πρόσθεσης. Μεταξύ των σταδίων προστέθηκαν pipeline registers (r1, r2, r3, r4) για τους εξής λόγους:

- **r1 (6 bits)**: Καταχωρεί τα bits εισόδου (b3, a3, b2, a2, b1, a1) που δεν έχουν ακόμη επεξεργαστεί, ώστε να είναι διαθέσιμα στο σωστό στάδιο pipeline.

- **r2 (5 bits)**: Καταχωρεί τα bits (b3, a3, b2, a2) καθώς και το αποτέλεσμα s0 από το FA0, ευθυγραμμίζοντάς τα χρονικά με το FA1.
- **r3 (4 bits)**: Καταχωρεί τα bits (b3, a3) και τα αποτελέσματα (s1, s0) για ευθυγράμμιση με το FA2.
- **r4 (3 bits)**: Καταχωρεί τα αποτελέσματα (s2, s1, s0) για ευθυγράμμιση με το FA3.

Οι pipeline registers είναι απαραίτητοι γιατί κάθε Full Adder χρειάζεται ένα κύκλο ρολογιού για να υπολογίσει το αποτέλεσμά του. Χωρίς τους ενδιάμεσους καταχωρητές, τα bits εισόδου των ανώτερων βαθμίδων θα είχαν ήδη αλλάξει τιμή πριν χρησιμοποιηθούν, καθώς νέα δεδομένα εισέρχονται σε κάθε κύκλο.

b) Κώδικας VHDL

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity pipelined_adder is
    port (
        i_clk  : in  std_logic;
        i_a    : in  std_logic_vector(3 downto 0);
        i_b    : in  std_logic_vector(3 downto 0);
        i_cin  : in  std_logic;
        o_sum  : out std_logic_vector(3 downto 0);
        o_cout : out std_logic
    );
end pipelined_adder;

architecture Behavioral of pipelined_adder is

    component sync_full_adder is
        port (
            i_clk  : in  std_logic;
            i_a    : in  std_logic;
            i_b    : in  std_logic;
            i_cin  : in  std_logic;
            o_sum  : out std_logic;
            o_cout : out std_logic
        );
    end component;

    -- Carry signals between FA stages
    signal c1, c2, c3 : std_logic;

    -- Sum outputs from each FA
```

```
    signal s_int : std_logic_vector(3 downto 0);

    -- Pipeline registers
    signal r1 : std_logic_vector(5 downto 0) := (others => '0');
    signal r2 : std_logic_vector(4 downto 0) := (others => '0');
    signal r3 : std_logic_vector(3 downto 0) := (others => '0');
    signal r4 : std_logic_vector(2 downto 0) := (others => '0');

begin

    -- Pipeline register process
    process(i_clk)
    begin
        if rising_edge(i_clk) then
            r1 <= i_b(3) & i_a(3) & i_b(2) & i_a(2) & i_b(1) & i_a(1);
            r2 <= r1(5) & r1(4) & r1(3) & r1(2) & s_int(0);
            r3 <= r2(4) & r2(3) & s_int(1) & r2(0);
            r4 <= s_int(2) & r3(1) & r3(0);
        end if;
    end process;

    FA0: sync_full_adder
        port map (
            i_clk => i_clk, i_a => i_a(0), i_b => i_b(0),
            i_cin => i_cin, o_sum => s_int(0), o_cout => c1
        );

    FA1: sync_full_adder
        port map (
            i_clk => i_clk, i_a => r1(0), i_b => r1(1),
            i_cin => c1, o_sum => s_int(1), o_cout => c2
        );

    FA2: sync_full_adder
        port map (
            i_clk => i_clk, i_a => r2(1), i_b => r2(2),
            i_cin => c2, o_sum => s_int(2), o_cout => c3
        );

    FA3: sync_full_adder
        port map (
            i_clk => i_clk, i_a => r3(2), i_b => r3(3),
            i_cin => c3, o_sum => s_int(3), o_cout => o_cout
        );

    -- Align all sums
    o_sum(0) <= r4(0);
```

```
    o_sum(1) <= r4(1);
    o_sum(2) <= r4(2);
    o_sum(3) <= s_int(3);

end Behavioral;
```

Testbench

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity pipelined_adder_tb is
end entity;

architecture tb of pipelined_adder_tb is

    component pipelined_adder is
        port (
            i_clk  : in  std_logic;
            i_a    : in  std_logic_vector(3 downto 0);
            i_b    : in  std_logic_vector(3 downto 0);
            i_cin  : in  std_logic;
            o_sum  : out std_logic_vector(3 downto 0);
            o_cout : out std_logic
        );
    end component;

    signal i_clk  : std_logic := '0';
    signal i_a    : std_logic_vector(3 downto 0) := (others => '0');
    signal i_b    : std_logic_vector(3 downto 0) := (others => '0');
    signal i_cin  : std_logic := '0';
    signal o_sum  : std_logic_vector(3 downto 0);
    signal o_cout : std_logic;

    constant CLK_PERIOD : time := 10 ns;

begin

    DUT: pipelined_adder
        port map (
            i_clk => i_clk, i_a => i_a, i_b => i_b,
            i_cin => i_cin, o_sum => o_sum, o_cout => o_cout
        );

    CLK_PROCESS: process
```

```
    begin
        i_clk <= '0';
        wait for CLK_PERIOD / 2;
        i_clk <= '1';
        wait for CLK_PERIOD / 2;
    end process;

    STIMULUS: process
        variable expected_cout : std_logic;
    begin
        for a in 0 to 15 loop
            for b in 0 to 15 loop
                for cin in 0 to 1 loop
                    i_a <= std_logic_vector(to_unsigned(a, 4));
                    i_b <= std_logic_vector(to_unsigned(b, 4));
                    if cin = 1 then
                        i_cin <= '1';
                    else
                        i_cin <= '0';
                    end if;
                    wait for 4 * CLK_PERIOD;
                    if (a + b + cin) >= 16 then
                        expected_cout := '1';
                    else
                        expected_cout := '0';
                    end if;
                    assert (o_sum = std_logic_vector(
                            to_unsigned((a + b + cin) mod 16, 4)))
                        and (o_cout = expected_cout)
                        report "FAIL: " & integer'image(a) & "+"
                            & integer'image(b) & "+"
                            & integer'image(cin);
                end loop;
            end loop;
        end loop;
        report "All 512 tests passed!";
        wait;
    end process;

end architecture tb;
```

Ανάλυση Αποτελεσμάτων Testbench

Το testbench ελέγχει εξαντλητικά όλους τους $16 \times 16 \times 2 = 512$ δυνατούς συνδυασμούς εισόδου. Για κάθε συνδυασμό, εφαρμόζονται οι είσοδοι και μετά από 4 κύκλους ρολογιού (η καθυστέρηση pipeline $T_{latency}$), ελέγχεται ότι το αποτέλεσμα είναι ορθό μέσω assert. Εφόσον

δεν υπήρξε κάποιο assert error κατά την εκτέλεση, επιβεβαιώθηκε η ορθή λειτουργία του κυκλώματος για όλες τις δυνατές εισόδους.

c) Κρίσιμο Μονοπάτι και Χρονική Ανάλυση (Critical Path & Timing)

Intra-Clock Paths - clk - Setup

Name	Slack ^1	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	7.940	1	2	2	FA0/o_cout_reg/C	r3_reg[1]_srl2/D	1.836	0.751	1.085
Path 2	8.288	0	1	1	r3_reg[0]_srl3/CLK	r4_reg[0]/D	1.617	1.617	0.000
Path 3	8.288	0	1	1	r3_reg[1]_srl2/CLK	r4_reg[1]/D	1.617	1.617	0.000
Path 4	8.346	1	2	2	r1_reg[1]/C	FA1/o_cout_reg/D	1.503	0.751	0.752
Path 5	8.346	1	2	2	r2_reg[2]/C	FA2/o_cout_reg/D	1.503	0.751	0.752
Path 6	8.346	1	2	2	FA1/o_cout_reg/C	FA2/o_sum_reg/D	1.503	0.751	0.752
Path 7	8.346	1	2	2	r3_reg[3]/C	FA3/o_cout_reg/D	1.503	0.751	0.752
Path 8	8.346	1	2	2	FA2/o_cout_reg/C	FA3/o_sum_reg/D	1.503	0.751	0.752
Path 9	8.795	0	1	1	r1_reg[2]/C	r2_reg[1]/D	0.790	0.456	0.334
Path 10	8.795	0	1	1	r1_reg[3]/C	r2_reg[2]/D	0.790	0.456	0.334

Σχήμα Κρίσιμο μονοπάτι του Pipelined Adder

Το κρίσιμο μονοπάτι του Pipelined Adder είναι από τον καταχωρητή FA0/o_cout_reg/C προς τον καταχωρητή r3_reg[1]_srl2/D με συνολική καθυστέρηση **2.060 ns** (clock period 10 ns, WNS = 7.940 ns). Το μονοπάτι διέρχεται από 1 επίπεδο λογικής και 2 routes. Πρόκειται για μονοπάτι register-to-register εντός ενός pipeline stage — ακριβώς αυτό που επιτυγχάνει η τεχνική pipeline.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	7.940 ns	Worst Hold Slack (WHS):	0.139 ns	Worst Pulse Width Slack (WPWS):	4.020 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	15	Total Number of Endpoints:	15	Total Number of Endpoints:	24

All user specified timing constraints are met.

Σχήμα Design Timing Summary του Pipelined Adder

Στο Design Timing Summary (Σχήμα) παρατηρούμε ότι όλα τα timing constraints ικανοποιούνται:

- **Setup**: WNS = 7.940 ns, δηλαδή $T_{clk} - T_{critical} = 10.0 - 2.060 = 7.940$ ns slack. Αυτό σημαίνει ότι το κύκλωμα θα μπορούσε να λειτουργήσει σε συχνότητα έως $f_{max} = 1/2.060$ ns $\approx$ 485 MHz.
- **Hold**: WHS = 0.139 ns > 0, άρα δεν υπάρχουν hold violations.
- **Pulse Width**: WPWS = 4.020 ns > 0, εντός προδιαγραφών.
- Σύνολο 15 setup endpoints και 24 pulse width endpoints, 0 failing — πλήρης επιτυχία.

Κατανάλωση Πόρων (Resource Utilization)

Name	Slice LUTs (17600)	Slice Registers (35200)	Bonded IOB (100)	BUFGCTRL (32)
pipelined_adder	10	21	15	1
FA0 (sync_full_adder)	2	1	0	0
FA1 (sync_full_adder_0)	2	1	0	0
FA2 (sync_full_adder_1)	2	2	0	0
FA3 (sync_full_adder_2)	1	2	0	0

Σχήμα Resource Utilization του Pipelined Adder

Από τον πίνακα utilization (Σχήμα) προκύπτει ότι ο Pipelined Adder χρησιμοποιεί συνολικά **10 Slice LUTs**, **21 Slice Registers** (flip-flops), **15 Bonded IOBs** και **1 BUFGCTRL**. Η κατανομή ανά instance είναι:

- Κάθε sync_full_adder (FA0--FA3) χρησιμοποιεί 2 LUTs (ένα για το sum, ένα για το carry) και 1-2 flip-flops (για τις εξόδους o_sum και o_cout).
- Οι FA2 και FA3 έχουν 2 registers αντί για 1, καθώς τα carry bits τους περνούν μέσω pipeline registers στα επόμενα stages.
- Τα υπόλοιπα flip-flops ($21 - 6 = 15$) αντιστοιχούν στους pipeline registers r1--r4 που ευθυγραμμίζουν τα ενδιάμεσα αποτελέσματα μεταξύ stages.

Ερώτημα 3: Πολλαπλασιαστής 4-bit Pipeline με Διάδοση Κρατουμένου

Περιγραφή Κυκλώματος

Ο Pipeline RCM (Ripple Carry Multiplier) 4-bit υλοποιεί τον πολλαπλασιασμό δύο 4-bit αριθμών με χρήση τεχνικής pipeline για την επίτευξη υψηλού throughput. Η αρχιτεκτονική βασίζεται στον μετασχηματισμό του παράλληλου πολλαπλασιασμού σε διαδοχικά στάδια πρόσθεσης.

Η δομή του κυκλώματος αποτελείται από:

- **3 Σύγχρονους Παράλληλους Αθροιστές**: Κάθε αθροιστής προσθέτει ένα μερικό γινόμενο (partial product) στο συσσωρευμένο άθροισμα. Ο πρώτος αθροιστής υπολογίζει το άθροιστα $A \times B_0 + A \times B_1$, ο δεύτερος προσθέτει το $A \times B_2$ στο αποτέλεσμα του πρώτου, και ο τρίτος προσθέτει το $A \times B_3$. Για τους παράλληλους αθροιστές χρειάζεται να ορίσουμε τα components τους καθώς και το component του full adder.
- **Καταχωρητές Pipeline**: Μεταξύ κάθε σταδίου προστίθενται καταχωρητές (registers) για την αποθήκευση:

- Των ενδιάμεσων αποτελεσμάτων προσθέσεων
- Των τιμών εισόδου A και B
- Των LSB bits που μεταφέρονται στο επόμενο στάδιο

- **Τελικός Πολλαπλασιασμός**: Το τελικό bit B_3 πολλαπλασιάζεται απευθείας με την είσοδο A και προστίθεται στο τελικό αποτέλεσμα χωρίς επιπλέον pipeline stage.

Η αρχιτεκτονική αυτή επιτυγχάνει **3-κύκλο latency**, καθώς το αποτέλεσμα εξέρχεται μετά από 3 περιόδους ρολογιού από την είσοδο των δεδομένων.

Full Adder Component

```
-----------------------------------------------
-- Synchronous Full Adder (Sequential)
-----------------------------------------------

library IEEE;
use IEEE.std_logic_1164.all;
entity FullAdder is
    port (
        i_A     : in    std_logic;
        i_B     : in    std_logic;
        i_Cin   : in    std_logic;
        o_Sum   : out   std_logic;
        o_Cout  : out   std_logic
    );
end FullAdder;

architecture rtl of FullAdder is
begin
    o_Sum <= i_A XOR i_B XOR i_Cin;
    o_Cout <= (i_A AND i_B) OR (i_Cin AND (i_A XOR i_B));
end rtl;
```

Parallel 4-bit Adder Component

```
-----------------------------------------------
-- Synchronous Parallel Adder
-----------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity SynchronousParallelAdder is
```

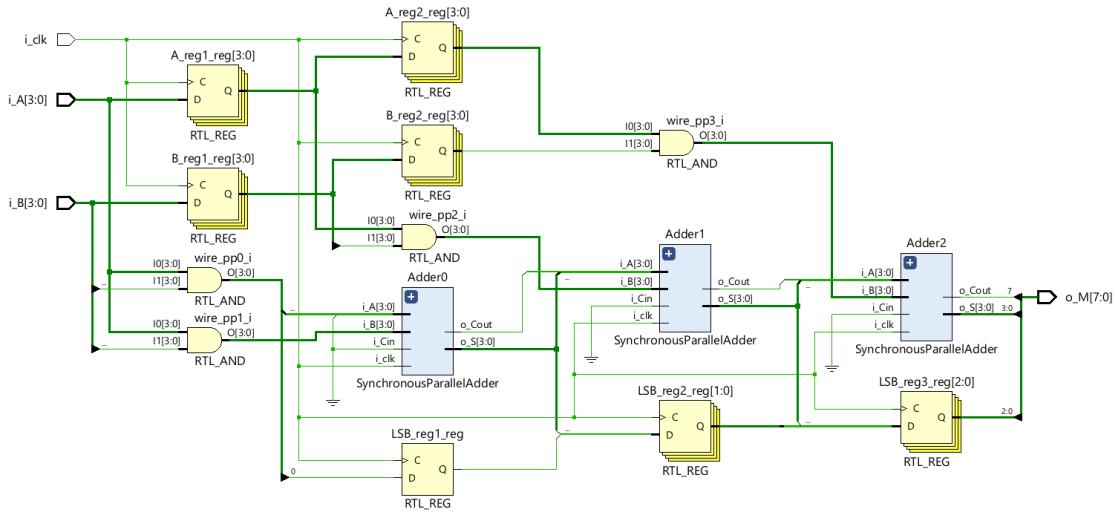
```
    Port (
        i_clk   : in  STD_LOGIC;
        i_A     : in  STD_LOGIC_VECTOR (3 downto 0);
        i_B     : in  STD_LOGIC_VECTOR (3 downto 0);
        i_Cin   : in  STD_LOGIC;
        o_S     : out STD_LOGIC_VECTOR (3 downto 0);
        o_Cout  : out STD_LOGIC
    );
end SynchronousParallelAdder;

architecture Behavioral of SynchronousParallelAdder is
    component FullAdder is
        Port ( i_a     : in  STD_LOGIC;
               i_b     : in  STD_LOGIC;
               i_cin   : in  STD_LOGIC;
               o_sum   : out STD_LOGIC;
               o_cout  : out STD_LOGIC);
    end component;

    signal carry      : STD_LOGIC_VECTOR(3 downto 0);

    signal wire_sum   : STD_LOGIC_VECTOR(3 downto 0);
    signal wire_cout  : STD_LOGIC;

begin
    FA0: FullAdder port map (i_a => i_A(0), i_b => i_B(0), i_cin =>
        i_Cin, o_sum => wire_sum(0), o_cout => carry(0));
    FA1: FullAdder port map (i_a => i_A(1), i_b => i_B(1), i_cin =>
        carry(0), o_sum => wire_sum(1), o_cout => carry(1));
    FA2: FullAdder port map (i_a => i_A(2), i_b => i_B(2), i_cin =>
        carry(1), o_sum => wire_sum(2), o_cout => carry(2));
    FA3: FullAdder port map (i_a => i_A(3), i_b => i_B(3), i_cin =>
        carry(2), o_sum => wire_sum(3), o_cout => carry(3));

    wire_cout <= carry(3);

    process(i_clk)
    begin
        if rising_edge(i_clk) then
            -- On clock tick, capture the results in the output
                registers
            o_S    <= wire_sum;
            o_Cout <= wire_cout;
        end if;
    end process;

end Behavioral;
```

Synchronous Pipelined 4-bit Ripple Carry Multiplier

```
-----------------------------------------------------
-- Synchronous Pipelined 4-bit Ripple Carry Multiplier
-----------------------------------------------------

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity Pipelined_RCM_4bit is
    Port (
        i_clk   : in  STD_LOGIC;
        i_A     : in  STD_LOGIC_VECTOR (3 downto 0);
        i_B     : in  STD_LOGIC_VECTOR (3 downto 0);
        o_M     : out STD_LOGIC_VECTOR (7 downto 0)
    );
end Pipelined_RCM_4bit;
architecture Structural of Pipelined_RCM_4bit is

    component SynchronousParallelAdder is
        Port (
            i_clk   : in  STD_LOGIC;
            i_A     : in  STD_LOGIC_VECTOR (3 downto 0);
            i_B     : in  STD_LOGIC_VECTOR (3 downto 0);
            i_Cin   : in  STD_LOGIC;
            o_S     : out STD_LOGIC_VECTOR (3 downto 0);
            o_Cout  : out STD_LOGIC
        );

    end component;
    -- Intermediate signals (wires)
    signal wire_pp0, wire_pp1, wire_pp2, wire_pp3  : STD_LOGIC_VECTOR(3 downto 0);
    signal adder_in_A1, adder_in_A2, adder_in_A3   : STD_LOGIC_VECTOR(3 downto 0);

    -- Pipeline delay registers for A and B (these hold the state of the inputs as the clock ticks)
    signal A_reg1, A_reg2  : STD_LOGIC_VECTOR(3 downto 0);
    signal B_reg1, B_reg2  : STD_LOGIC_VECTOR(3 downto 0);

    -- Pipeline delay registers for the LSBs of the result
    signal LSB_reg1  : STD_LOGIC;
    signal LSB_reg2  : STD_LOGIC_VECTOR(1 downto 0);
    signal LSB_reg3  : STD_LOGIC_VECTOR(2 downto 0);

```

```vhdl
    -- Adder outputs
    signal sum1, sum2, sum3 : STD_LOGIC_VECTOR(3 downto 0);
    signal c1, c2, c3       : STD_LOGIC;

begin
    -- Note to self: The combinatorial logic is better to be seperated from the sequential logic
    -- because that way it is easier to understand and implement the design

    -- Combinatorial Part (generate the inputs for the adders)
    wire_pp0 <= i_A     and (3 downto 0 => i_B(0));
    wire_pp1 <= i_A     and (3 downto 0 => i_B(1));
    wire_pp2 <= A_reg1 and (3 downto 0 => B_reg1(2));
    wire_pp3 <= A_reg2 and (3 downto 0 => B_reg2(3));

    -- Adder inputs finalized
    adder_in_A1 <= '0' & wire_pp0(3 downto 1);
    adder_in_A2 <= c1  & sum1(3 downto 1);
    adder_in_A3 <= c2  & sum2(3 downto 1);

    -- Synchronous adder logic which create the partial products
    Adder0: SynchronousParallelAdder port map (
        i_clk => i_clk, i_A => adder_in_A1, i_B => wire_pp1, i_Cin => '0', o_S => sum1, o_Cout => c1
    );
    Adder1: SynchronousParallelAdder port map (
        i_clk => i_clk, i_A => adder_in_A2, i_B => wire_pp2, i_Cin => '0',  o_S => sum2, o_Cout => c2
    );
    Adder2: SynchronousParallelAdder port map (
        i_clk => i_clk, i_A => adder_in_A3, i_B => wire_pp3, i_Cin => '0',  o_S => sum3, o_Cout => c3
    );

    -- Synchronous logic for the pipeline delay registers
    process(i_clk)
    begin
        if rising_edge(i_clk) then
            A_reg1 <= i_A;
            A_reg2 <= A_reg1;

            B_reg1 <= i_B;
            B_reg2 <= B_reg1;

            -- Delay the LSBs so they reach the final output at the exact same time
```

```
            LSB_reg1                <= wire_pp0(0);
            LSB_reg2(1 downto 0) <= sum1(0) & LSB_reg1;
            LSB_reg3(2 downto 0) <= sum2(0) & LSB_reg2(1 downto 0);

        end if;
    end process;
    o_M <= c3 & sum3 & LSB_reg3;

end Structural;
```



Σχήμα RTL δομή του Pipeline RCM 4-bit πολλαπλασιαστή

Περιγραφή Testbench

Το testbench Pipelined_RCM_tb.vhd εκτελεί εξαντλητικό έλεγχο (exhaustive testing) επαληθεύοντας όλους τους 256 συνδυασμούς εισόδων ($2^4 \times 2^4 = 256$). Οι κύριες λειτουργίες του είναι:

- **Παραγωγή Ρολογιού**: Σήμα ρολογιού με περίοδο 10 ns (100 MHz)
- **Διπλός Βρόχος**: Δύο εμφωλευμένοι βρόχοι επαναλαμβάνουν για όλες τις τιμές A (0-15) και B (0-15)
- **Χειρισμός Latency**: Λόγω του pipeline των 3 αθροιστών, τα αποτελέσματα καθυστερούν 3 κύκλους. Χρησιμοποιείται πίνακας delay_array για την αποθήκευση των αναμενόμενων τιμών με την κατάλληλη καθυστέρηση
- **Assertions**: Για κάθε κύκλο μετά την αρχικοποίηση, ελέγχεται η ορθότητα του αποτελέσματος με assertions
- **Flush**: Μετά την ολοκλήρωση των βρόχων, απαιτούνται 3 επιπλέον κύκλοι για την εκκαθάριση (flush) των δεδομένων που βρίσκονται ακόμα στο pipeline

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tb_Pipelined_RCM_4bit is
end tb_Pipelined_RCM_4bit;

architecture Behavioral of tb_Pipelined_RCM_4bit is

    -- Component Declaration for the device being tested
    component Pipelined_RCM_4bit is
        Port (
            i_clk   : in  STD_LOGIC;
            i_A     : in  STD_LOGIC_VECTOR (3 downto 0);
            i_B     : in  STD_LOGIC_VECTOR (3 downto 0);
            o_M     : out STD_LOGIC_VECTOR (7 downto 0)
        );
    end component;

    -- Testbench Signals
    signal clk    : STD_LOGIC := '0';
    signal sig_A  : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal sig_B  : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal sig_M  : STD_LOGIC_VECTOR(7 downto 0);

    constant clk_period : time := 10 ns;

    -- Verification Variables
    -- Array to store the expected result by 3 cycles
    type delay_array is array (1 to 3) of integer;
    signal expected_pipe : delay_array := (others => 0);

    -- Counter to track cycles
    signal test_cycle : integer := 0;

begin

    -- Device instance
    DUT: Pipelined_RCM_4bit port map (
        i_clk => clk,
        i_A   => sig_A,
        i_B   => sig_B,
        o_M   => sig_M
    );

    -- Clock generation
    clk_process : process
```

```
    begin
        clk <= '0';
        wait for clk_period / 2;
        clk <= '1';
        wait for clk_period / 2;
    end process;

    -- Exhaustive stimulus with assertions
    stimulus: process
    begin

        -- Check all possible combinations for (A,B)
        for i in 0 to 15 loop
            for j in 0 to 15 loop

                wait until falling_edge(clk);

                -- Check the result after the 3-cycle pipeline latency
                -- Therefore only start checking after 3 cycles have passed
                if test_cycle >= 3 then
                    assert to_integer(unsigned(sig_M)) = expected_pipe(3)
                    report "Mismatched result! " &
                           "Expected: " & integer'image(expected_pipe(3)) &
                           ", Got: " & integer'image(to_integer(unsigned(sig_M)))
                    severity error;
                end if;

                -- Delay the expected results by 3 cycles
                expected_pipe(3) <= expected_pipe(2);
                expected_pipe(2) <= expected_pipe(1);
                expected_pipe(1) <= i * j;

                -- Store the new inputs
                sig_A <= std_logic_vector(to_unsigned(i, 4));
                sig_B <= std_logic_vector(to_unsigned(j, 4));

                test_cycle <= test_cycle + 1;

            end loop;
        end loop;

        -- After the loops finish, the last 3 multiplications are still stuck inside the pipeline.
```

```
        -- So we need 3 more clock cycles to get them
        for k in 1 to 3 loop
            wait until falling_edge(clk);

            -- Verify the flushing data
            assert to_integer(unsigned(sig_M)) = expected_pipe(3)
            report "Mismatched result during flush! Expected: " &
               integer'image(expected_pipe(3))
            severity error;

            -- Delay the expected results
            expected_pipe(3) <= expected_pipe(2);
            expected_pipe(2) <= expected_pipe(1);
            expected_pipe(1) <= 0;

            sig_A <= (others => '0');
            sig_B <= (others => '0');
        end loop;

        -- Stop the simulation
        assert false report "Exhaustive Test Completed Successfully!
           All 256 combinations passed." severity failure;

    end process;

end Behavioral;
```

Η επιτυχής ολοκλήρωση του testbench επιβεβαιώνεται με την απουσία assertion errors.

Κρίσιμο Μονοπάτι και Χρονική Καθυστέρηση

Το κρίσιμο μονοπάτι στον πολλαπλασιαστή Pipeline RCM διέρχεται μέσω ενός σταδίου πρόσθεσης, δηλαδή μέσω ενός Σύγχρονου Παράλληλου Αθροιστή. Η χρονική καθυστέρηση υπολογίζεται ως:

$$t_{cp} = t_{co_FF} + t_{logic_FA} + t_{routing}$$

Όπου:

- t_{co_FF}: Χρόνος clock-to-output του flip-flop
- t_{logic_FA}: Χρονική καθυστέρηση ενός full adder (περιλαμβάνει τη λογική AND για τη δημιουργία partial product και την πρόσθεση)
- $t_{routing}$: Χρόνος δρομολόγησης μεταξύ κυκλωμάτων

Name	Slack ^1	Levels	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 1	7.410	1	6	B_reg1_reg[2]/C	Adder1/o_S_reg[1]/D	2.259	0.802	1.457	10.0	sys_clk	sys_clk
Path 2	7.633	2	6	B_reg1_reg[2]/C	Adder1/o_S_reg[3]/D	2.335	0.897	1.438	10.0	sys_clk	sys_clk
Path 3	7.655	2	6	B_reg1_reg[2]/C	Adder1/o_S_reg[2]/D	2.316	0.897	1.419	10.0	sys_clk	sys_clk
Path 4	7.927	2	3	Adder1/o_S_reg[1]/C	Adder2/o_Cout_reg/D	2.089	1.069	1.020	10.0	sys_clk	sys_clk
Path 5	7.933	2	3	Adder1/o_S_reg[1]/C	Adder2/o_S_reg[3]/D	2.085	1.069	1.016	10.0	sys_clk	sys_clk
Path 6	7.934	2	3	Adder1/o_S_reg[1]/C	Adder2/o_S_reg[2]/D	2.086	1.069	1.017	10.0	sys_clk	sys_clk
Path 7	7.971	1	6	B_reg1_reg[2]/C	LSB_reg3_reg[2]_srl2/D	1.923	0.773	1.150	10.0	sys_clk	sys_clk
Path 8	7.995	1	3	Adder1/o_S_reg[1]/C	Adder2/o_S_reg[1]/D	1.863	0.715	1.148	10.0	sys_clk	sys_clk
Path 9	8.108	2	6	B_reg1_reg[2]/C	Adder1/o_Cout_reg/D	1.909	0.897	1.012	10.0	sys_clk	sys_clk
Path 10	8.300	0	3	A_reg1_reg[3]/C	A_reg2_reg[3]/D	1.400	0.478	0.922	10.0	sys_clk	sys_clk

Σχήμα Κρίσιμα μονοπάτια του Pipeline RCM

Από το implementation προκύπτει ότι το κρίσιμο μονοπάτι έχει χρονική καθυστέρηση **2.259 ns**.

Ερώτημα 4: Bonus Άσκηση εξοικείωσης με το Zybo

Σε αυτό το ερώτημα μας δίνεται ο κώδικας blinkled.vhd όπου χρησιμοποιείται ένας μετρητής, του οποίου το MSB ανάβει ένα LED στο Zybo. Ο μετρητής αυξάνεται σε κάθε κύκλο ρολογιού, και το MSB του θα αλλάζει κατάσταση (toggle) κάθε 2^{N-1} κύκλους, όπου N είναι το πλήθος των bits του μετρητή.

Τροποποιούμε αυτό το αρχείο ώστε να πραγματοποιήσουμε μία απεικόνιση και στα 4 leds της πλακέτας που προσομοιώνει την κίνηση ενός βαγονέτου, το οποίο πάει από δεξιά προς τα αριστερά και μετά ξανά προς τα δεξιά. Για να το πετύχουμε αυτό, αξιοποιούμε το MSB του μετρητή ως τρόπος ένδειξης της κατεύθυνσης, το δεύτερο MSB για την εναλλαγή μεταξύ του ζεύγους LEDs που βρίσκεται στο αριστερό μέρος και του ζεύγους που βρίσκεται στο δεξί μέρος και το τρίτο MSB για την επιλογή του LED που θα ανάψει μέσα στο ζεύγος. Με αυτόν τον τρόπο, δημιουργούμε το επιθυμητό εφέ κίνησης.

Κώδικας VHDL για το BlinkLED

```
-----------------------------------------------------------------
-- Blink Single LED Example
-- Digital VLSI Course
-- Microlab <ipapalambrou@microlab.ntua.gr> Ilias Papalamprou
-----------------------------------------------------------------
-- This code implements a simple module that blinks the 4 output
-- LEDs on Zybo, creating a wagon effect from left to right and back
-- using a single-bit shift register.
--
-- Original code from:
-- https://digilent.com/reference/learn/programmable-logic/tutorials/
   zybo-led-demo/start
-----------------------------------------------------------------
```

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use ieee.std_logic_unsigned.all;

entity blink_led is
    generic (
        COUNTER_LIMIT : natural := 27
    );
    port (
        ------------------------------------------------------------
        -- CONTROL SIGNALS
        i_clk    : in     std_logic;
        ------------------------------------------------------------
        -- MAIN SIGNALS
        o_led    : out    std_logic_vector(3 downto 0)
    );
end entity;

architecture behavioral_arch of blink_led is

    signal count_reg : std_logic_vector(COUNTER_LIMIT-1 downto 0) := (
        others => '0');
    signal a,b,dir : std_logic;
begin

    BLINK_LED_PROCESS : process(i_clk)
    begin
        if (rising_edge(i_clk)) then
            count_reg <= count_reg + 1;
        end if;
    end process;
    a <= count_reg(COUNTER_LIMIT-3);
    b <= count_reg(COUNTER_LIMIT-2);
    dir <= count_reg(COUNTER_LIMIT-1);
    o_led(0) <= (a and b)          when dir = '1' else (not a and not b)
        ;
    o_led(1) <= (not a and b)      when dir = '1' else (a and not b);
    o_led(2) <= (a and not b)      when dir = '1' else (not a and b);
    o_led(3) <= (not a and not b) when dir = '1' else (a and b);
end behavioral_arch;
```

Φυσικά, για να λειτουργήσει ο παραπάνω κώδικας, πρέπει να γίνει κατάλληλη σύνδεση των σημάτων εισόδου και εξόδου στο constraint file του Zybo, ώστε να ορίσουμε τη συχνότητα του ρολογιού και να συνδέσουμε τα LEDs με τα σωστά pins της πλακέτας.

Constraint File (XDC) για το Zybo

```
# Clock signal
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports
    { i_clk }];
create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [
   get_ports { i_clk }];

# LEDs
set_property -dict { PACKAGE_PIN M14   IOSTANDARD LVCMOS33 } [get_ports
    { o_led[0] }];
set_property -dict { PACKAGE_PIN M15   IOSTANDARD LVCMOS33 } [get_ports
    { o_led[1] }];
set_property -dict { PACKAGE_PIN G14   IOSTANDARD LVCMOS33 } [get_ports
    { o_led[2] }];
set_property -dict { PACKAGE_PIN D18   IOSTANDARD LVCMOS33 } [get_ports
    { o_led[3] }];
```